

Infosys SpringBoard 7.0 Virtual Internship



— Data Visualization —

PROJECT TITLE: _____

Optimizing IT Support Team Performance Using Analytics

— (Supportlytics) —



IT Support
Analytics



Performance
Analysis



Geographic
Insights



Team
Efficiency



Data Driven
Decisions



Presented by:

Yona Gochipathala



Table of Contents:

S. No.	Name of the Contents
1	Introduction
2	Problem Statement
3	Objectives
4	Dataset Description
5	Technologies Used
6	Module 1 – Project Initialization & Dataset Setup
7	Module 2 – Data Cleaning & Feature Engineering
8	Module 3 – Exploratory Visualization
9	Module 4 – Cluster & Similarity Analysis
10	Module 5 – Performance Trend Analysis
11	Module 6 – Geographic & Category-Level Insights
12	Module 7 – Power BI Dashboard
13	Key Findings
14	Recommendations
15	Expected Business Impact
16	Conclusion
17	GitHub Repository / Project Files

1.Introduction:

Supportlytics is an IT support analytics project developed as part of the Infosys Springboard 7.0 Internship. The project focuses on analyzing IT support ticket data to understand support-team performance, ticket patterns, resolution efficiency, customer satisfaction, and geographic trends.

The project follows an end-to-end data analytics workflow starting from dataset preparation and data cleaning, followed by exploratory data analysis, cluster and similarity analysis, performance analysis, geographic analysis, and interactive dashboard development using Power BI.

The analysis helps identify important operational patterns such as ticket volume by category and priority, resolution-time differences, unresolved high-priority tickets, regional ticket concentration, and relationships between ticket clusters and performance scores.

2.Problem Statement :

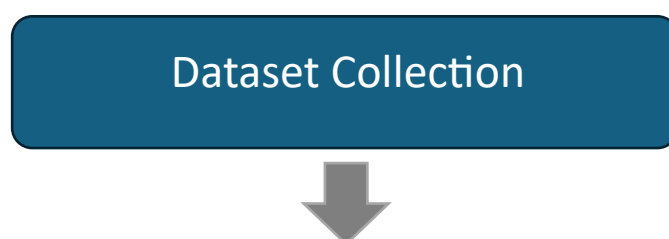
IT support teams generate a large number of tickets containing information about incidents, requests, problems, priorities, categories, resolution times, support teams, countries, clusters, and customer satisfaction.

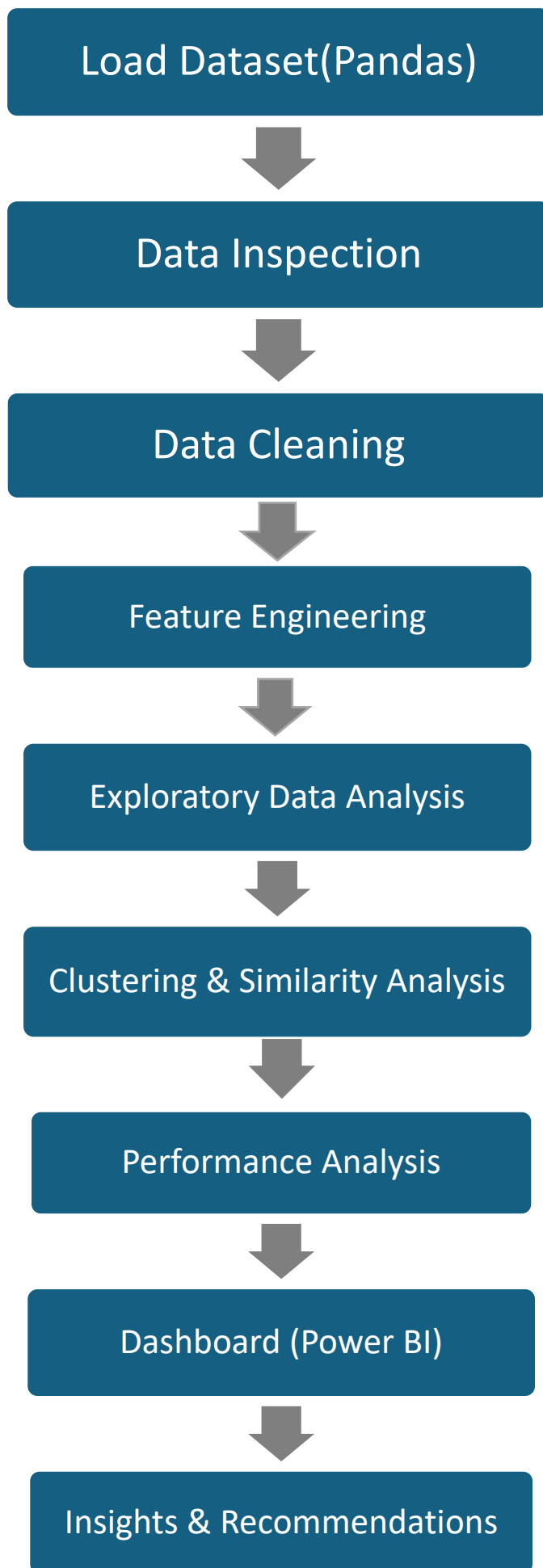
Without systematic analysis, it can be difficult to identify performance bottlenecks, high-priority unresolved tickets, overloaded support areas, regional ticket concentration, and problematic ticket clusters.

Therefore, Supportlytics aims to transform raw IT support ticket data into meaningful analytical insights through data cleaning, exploratory analysis, clustering analysis, performance analysis, geographic analysis, and interactive visualization.

3.Objectives

3.1 Project Workflow:





3.2 Project Objectives:

- Analyze the overall IT support ticket dataset.

- Clean and standardize the raw dataset.
- Create meaningful analytical features.
- Analyze ticket clusters and similarity scores.
- Understand ticket distribution by type, priority, category, and queue..
- Compare resolution times across priorities, ticket types, countries, and teams.
- Identify unresolved high-priority tickets.
- Analyze geographic ticket distribution.
- Understand issue categories across regions.
- Study cluster size versus performance score.
- Develop an interactive Power BI dashboard.
- Generate actionable insights and recommendations.

4. Dataset Description

I. Overview

The Supportlytics project uses a simulated IT Support Ticket Dataset created for internship analysis. The dataset contains **5,000 ticket records and 30 columns** covering ticket information, dates, priority, type, category, country, clustering information, support assignment, SLA status, and resolution information.

Important fields:

```
# Display all column names
print(df.columns)

# Number of columns
print("Total Columns:", len(df.columns))

Index(['Ticket_ID', 'Created_Date', 'Resolution_Date', 'Status', 'Priority',
      'Ticket_Type', 'Category', 'Country', 'Cluster_ID', 'Cluster_Name',
      'Similarity_Score', 'Similarity_Level', 'First_Response_Date',
      'Assigned_Agent', 'Team', 'SLA_Status', 'Reopened', 'Resolution_Code',
      'Department', 'Customer_ID', 'Channel', 'Description', 'Tags',
      'Product', 'Customer_Satisfaction', 'Escalation_Level', 'Region',
      'Device_Type', 'Operating_System', 'Resolution_Notes'],
      dtype='object')
Total Columns: 30
```

Dataset statistics:

- Total Records: **5,000**
- Columns: **30**
- Countries: **5**
- KPIs : **13**
- Ticket Types: Incident, Request, Problem
- Priority Levels: Low, Medium, High, Critical

II. KPIs:

Total 13 KPIs, There are, Total Tickets, Open/Closed Tickets, Average Resolution Time, SLA Compliance Rate, Customer Satisfaction, Category/Country analysis, Top Performing Agents and Cluster Similarity Score.

5. Technologies Used:

Technology	Purpose
Python	Data analysis and processing
Pandas	Data loading, cleaning and manipulation
NumPy	Numerical operations
Matplotlib	Data visualization
Plotly	Interactive visualization
Power BI	Interactive dashboard
Jupyter Notebook	Analysis and documentation
Excel	Dataset / processed-data storage
GitHub	Project version control and sharing

6. Module 1: Project Initialization & Dataset Setup

6.1 Project Initialization:

The first module focused on establishing the project objectives, defining key performance indicators, understanding the analytical workflow, and preparing the dataset for further analysis.

Activities performed:

- Defined project objectives.
- Defined KPIs.
- Established the analytical workflow.
- Loaded the dataset using Pandas.
- Inspected dataset structure.
- Examined data types.
- Checked missing values.
- Performed initial ticket distribution analysis.

6.2 Dataset Loading:

The Supportlytics dataset was loaded into a Pandas DataFrame for analysis. The dataset contained 5,000 records and 30 columns.

```
import pandas as pd
import numpy as np

df = pd.read_excel("Supportlytics_Complete_Dataset.xlsx")
print(df.shape)
```

```
(5000, 30)
```

6.3 Initial Data Exploration

- Dataset shape
- Column names
- Data types
- Missing values
- Basic ticket distributions

Data types

```
# Display data types of all columns
```

```
print(df.dtypes)
```

Ticket_ID	object
Created_Date	object
Resolution_Date	object
Status	object
Priority	object
Ticket_Type	object
Category	object
Country	object
Cluster_ID	int64
Cluster_Name	object
Similarity_Score	float64
Similarity_Level	object
First_Response_Date	object
Assigned_Agent	object
Team	object
SLA_Status	object
Reopened	object
Resolution_Code	object
Department	object
Customer_ID	object
Channel	object
Description	object
Tags	object
Product	object
Customer_Satisfaction	float64
Escalation_Level	object
Region	object
Device_Type	object
Operating_System	object
Resolution_Notes	object

dtype: object

Missing Values

```
# Check missing values in each column
```

```
print(df.isnull().sum())
```

Ticket_ID	0
Created_Date	0
Resolution_Date	1621
Status	0
Priority	0
Ticket_Type	0
Category	0
Country	0
Cluster_ID	0
Cluster_Name	0
Similarity_Score	0
Similarity_Level	0
First_Response_Date	0
Assigned_Agent	0
Team	0
SLA_Status	0
Reopened	0
Resolution_Code	0
Department	0
Customer_ID	0
Channel	0
Description	0
Tags	0
Product	0
Customer_Satisfaction	1621
Escalation_Level	0
Region	0
Device_Type	0
Operating_System	0
Resolution_Notes	1621

dtype: int64

6.4 Initial Ticket Distribution:

Calculate initial ticket distribution by Type, Priority, and Category

```
2]: # Ticket Distribution by Type
print(df['Ticket_Type'].value_counts())
```

```
Ticket_Type
Incident    1705
Request     1656
Problem     1639
Name: count, dtype: int64
```

```
3]: # Ticket Distribution by Priority
print(df['Priority'].value_counts())
```

```
Priority
Low      1284
Medium   1255
Critical 1233
High     1228
Name: count, dtype: int64
```

```
4]: # Ticket Distribution by Category
print(df['Category'].value_counts())
```

```
Category
Network    867
Hardware   857
Security   839
Software   833
Access     806
Email      798
Name: count, dtype: int64
```


7. Module 2: Data Cleaning & Feature Engineering

7.1 Data Quality Check:

A data quality assessment was performed before analysis. Missing values, duplicate records, data types, blank text fields, and inconsistent text formatting were examined.

Findings:

- **1,621 Resolution_Date values were missing**
- These corresponded to tickets that were still open.
- No duplicate ticket records were found.
- Some text fields contained extra spaces and blank values.

8. Data Cleaning

Cleaning Steps:

1. Checked missing values.
2. Removed duplicate records.
3. Removed extra whitespace.
4. Replaced blank values with "Unknown" where applicable.
5. Standardized text fields.
6. Rechecked data quality.
7. Saved cleaned dataset.

9. Feature Engineering

1. Resolution_Duration:

Resolution_Duration measures the time required to resolve a ticket. It was calculated as the difference between Resolution_Date and Created_Date and converted into hours.

$$\text{Resolution_Duration} = \text{Resolution_Date} - \text{Created_Date}$$

2. Priority_Score:

Machine learning models and analysis work better with numbers than text.

So, I converted priorities into numeric values.

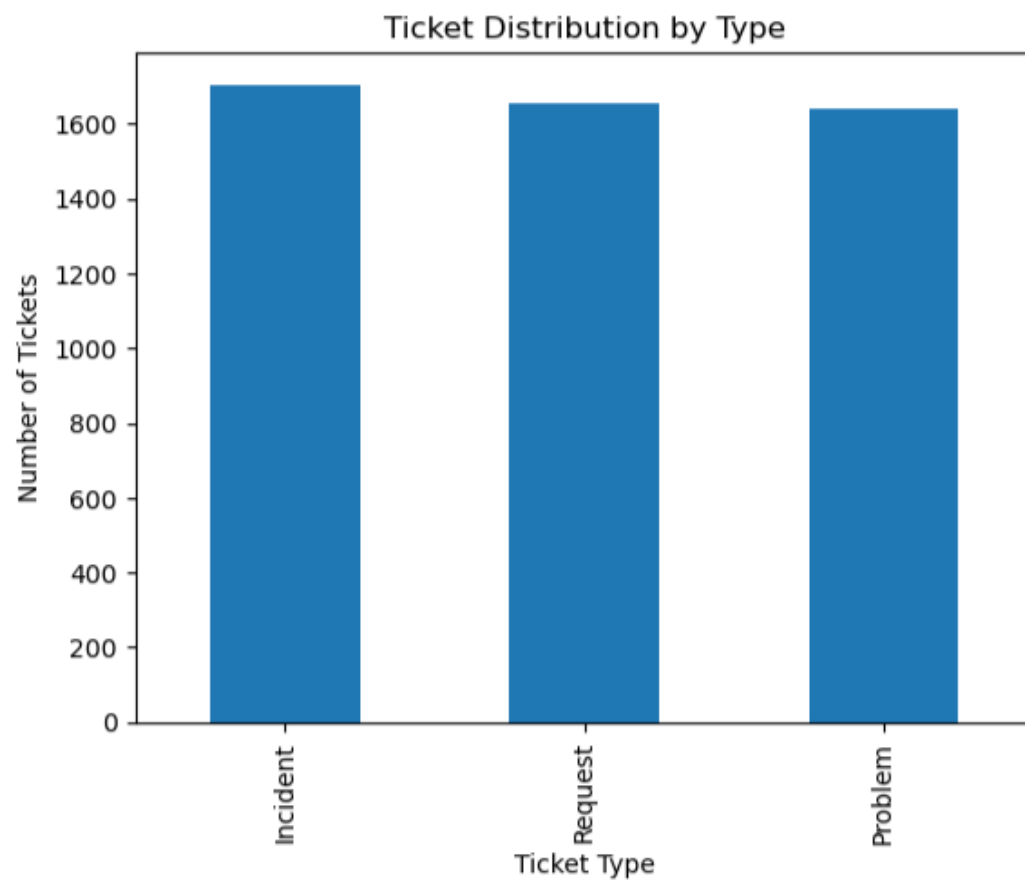
Priority	Score
Low	1
Medium	2
High	3
Critical	4

10. Module 3: Exploratory Visualization

10.1 Ticket Type Distribution

```
### Ticket Ticket Distribution by Type
import matplotlib.pyplot as plt

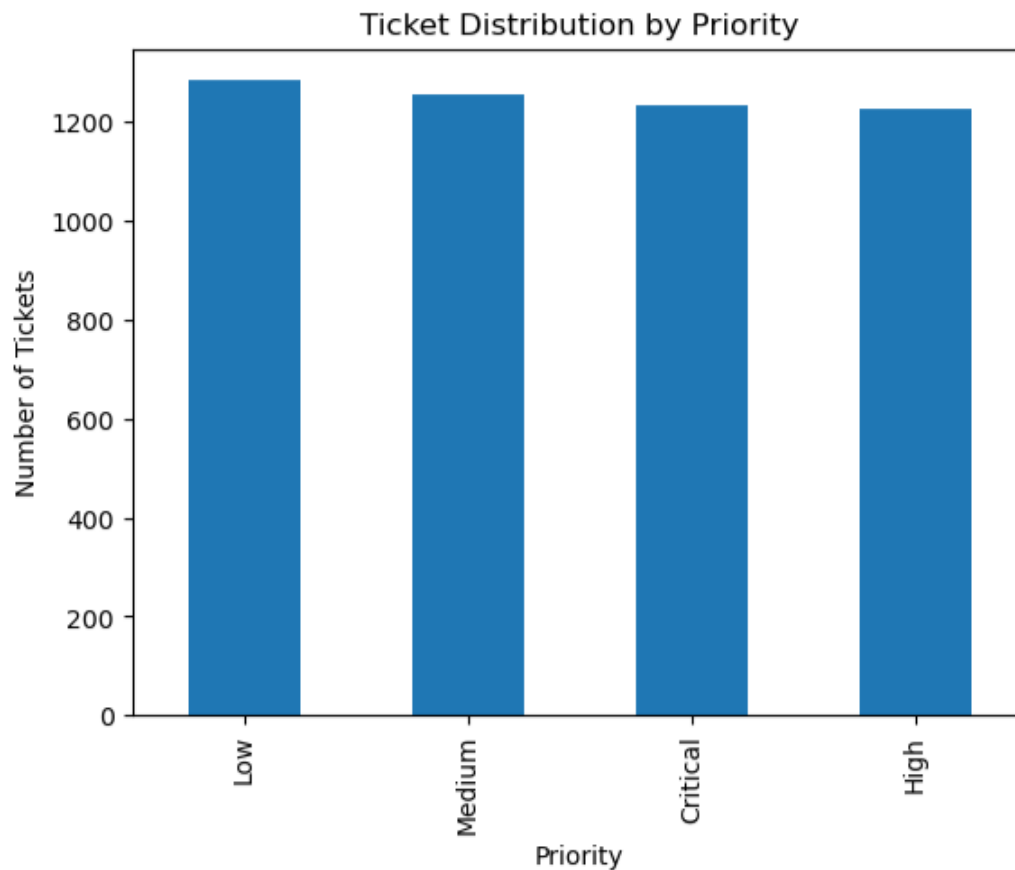
df["Ticket_Type"].value_counts().plot(kind="bar", title="Ticket Distribution by Type")
plt.xlabel("Ticket Type")
plt.ylabel("Number of Tickets")
plt.show()
```



10.2 Ticket Distribution by Priority

```
import matplotlib.pyplot as plt

df["Priority"].value_counts().plot(kind="bar", title="Ticket Distribution by Priority")
plt.xlabel("Priority")
plt.ylabel("Number of Tickets")
plt.show()
```



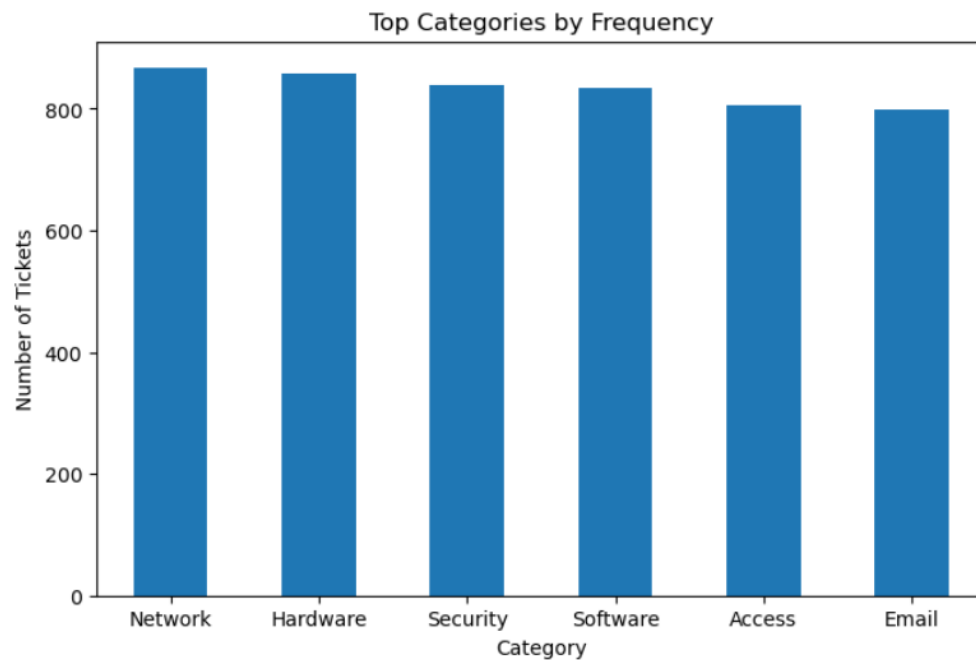
10.3 Ticket Distribution by Category

```
# Top Categories by Frequency
top_categories = df["Category"].value_counts()

plt.figure(figsize=(8,5))
top_categories.plot(kind="bar")

plt.title("Top Categories by Frequency")
plt.xlabel("Category")
plt.ylabel("Number of Tickets")

plt.xticks(rotation=0)
plt.show()
```



10.4 Cluster Distribution

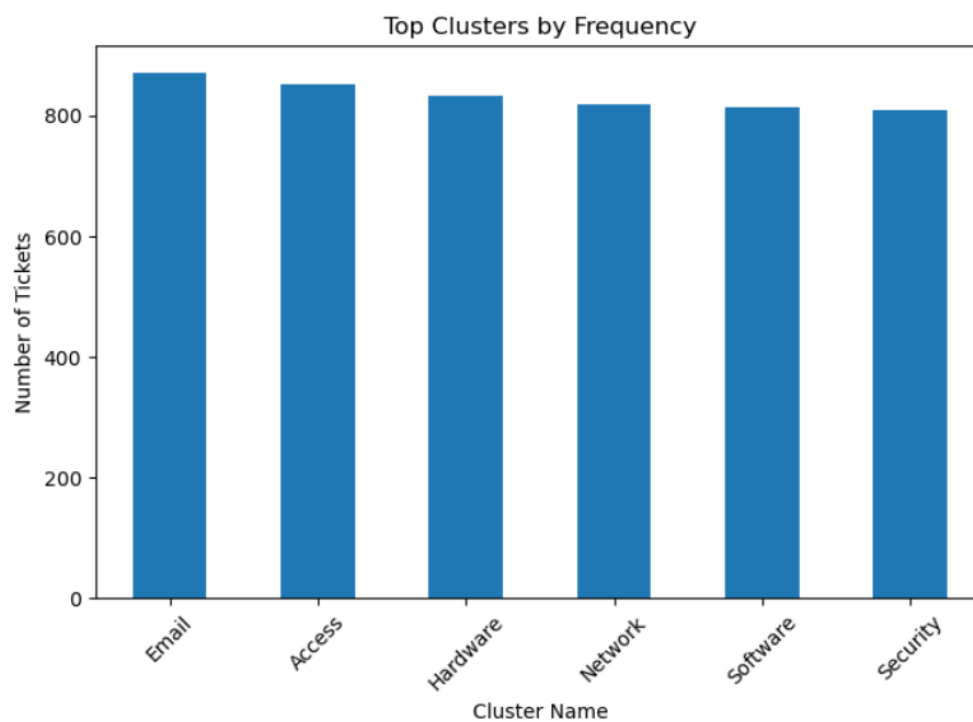
```
import matplotlib.pyplot as plt

top_clusters = df["Cluster_Name"].value_counts()

plt.figure(figsize=(8,5))
top_clusters.plot(kind="bar")

plt.title("Top Clusters by Frequency")
plt.xlabel("Cluster Name")
plt.ylabel("Number of Tickets")

plt.xticks(rotation=45)
plt.show()
```



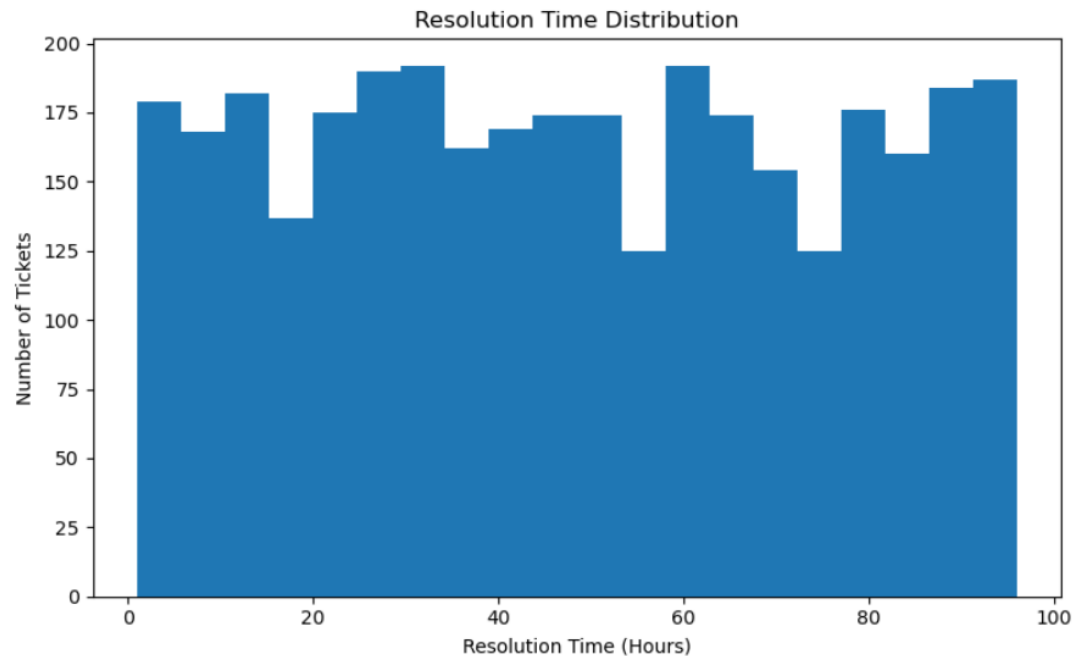
10.5 Resolution Time Distribution

```
plt.figure(figsize=(8,5))

plt.hist(df['Resolution_Time'], bins=20)

plt.title("Resolution Time Distribution")
plt.xlabel("Resolution Time (Hours)")
plt.ylabel("Number of Tickets")

plt.tight_layout()
plt.show()
```



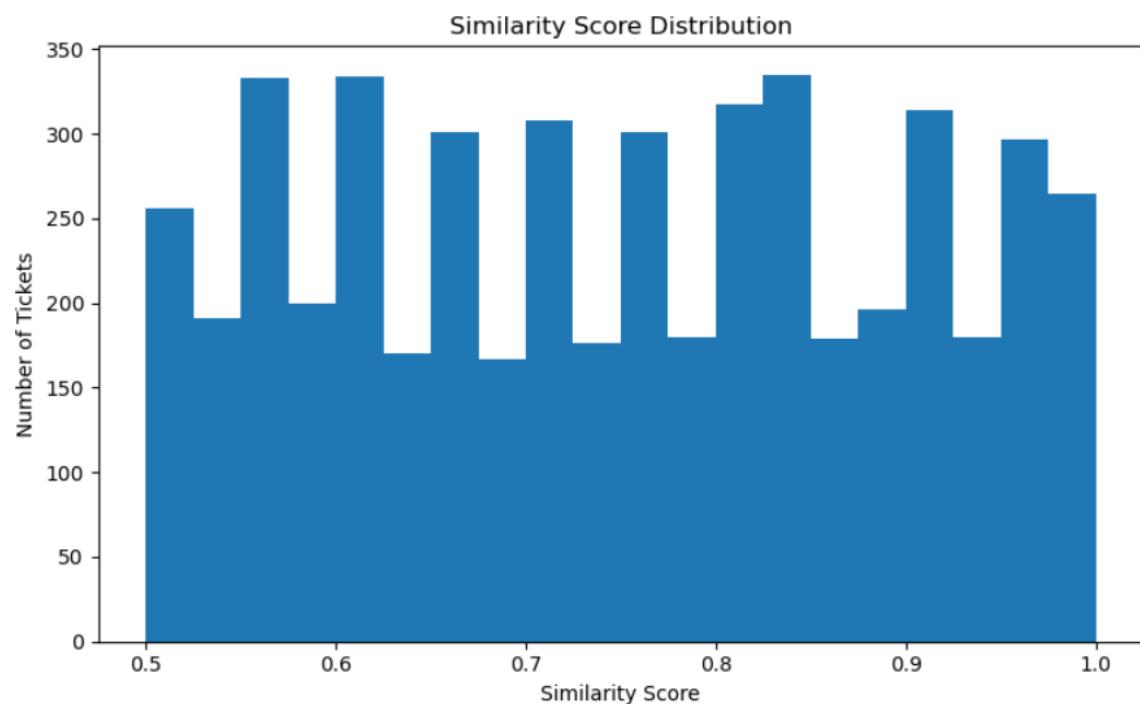
10.6 Similarity Score Distribution

```
plt.figure(figsize=(8,5))

plt.hist(df['Similarity_Score'], bins=20)

plt.title("Similarity Score Distribution")
plt.xlabel("Similarity Score")
plt.ylabel("Number of Tickets")

plt.tight_layout()
plt.show()
```



10.7 Priority Distribution

```
priority_counts = df['Priority'].value_counts()

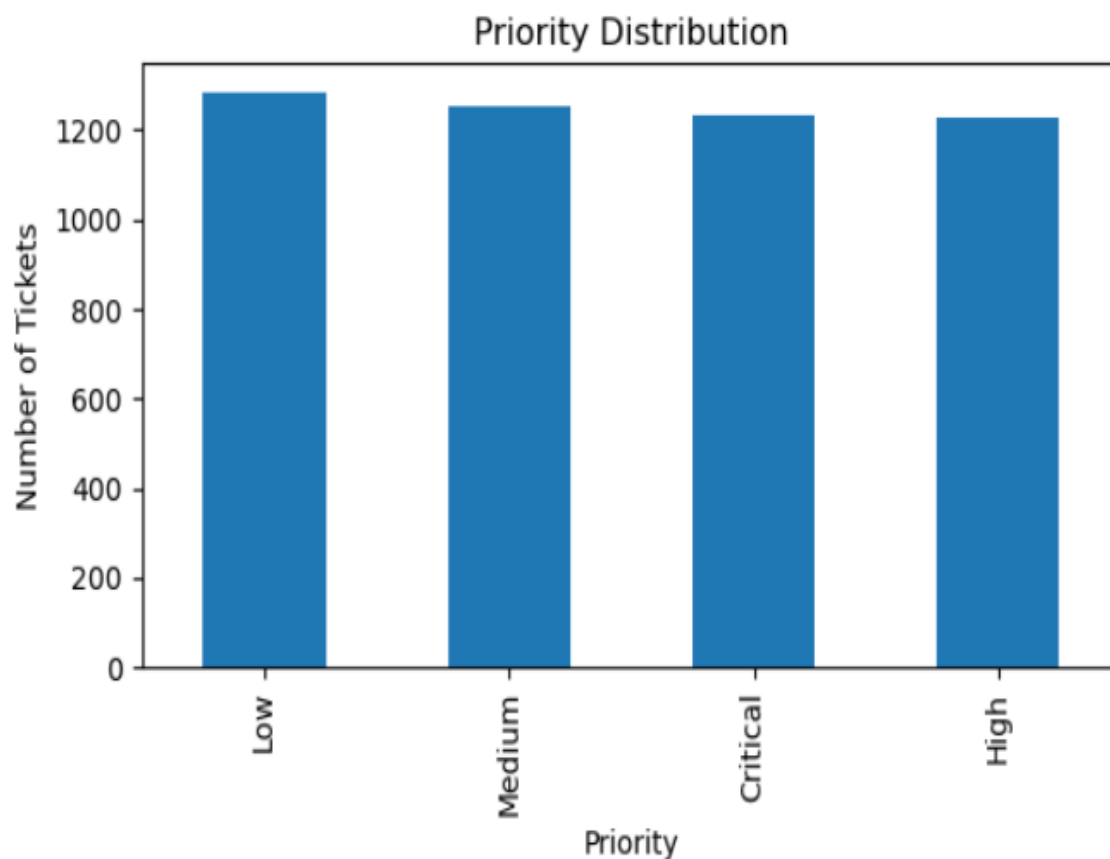
print(priority_counts)

plt.figure(figsize=(6,4))
priority_counts.plot(kind='bar')

plt.title("Priority Distribution")
plt.xlabel("Priority")
plt.ylabel("Number of Tickets")

plt.tight_layout()
plt.show()
```

```
Priority
Low      1284
Medium   1255
Critical 1233
High     1228
Name: count, dtype: int64
```

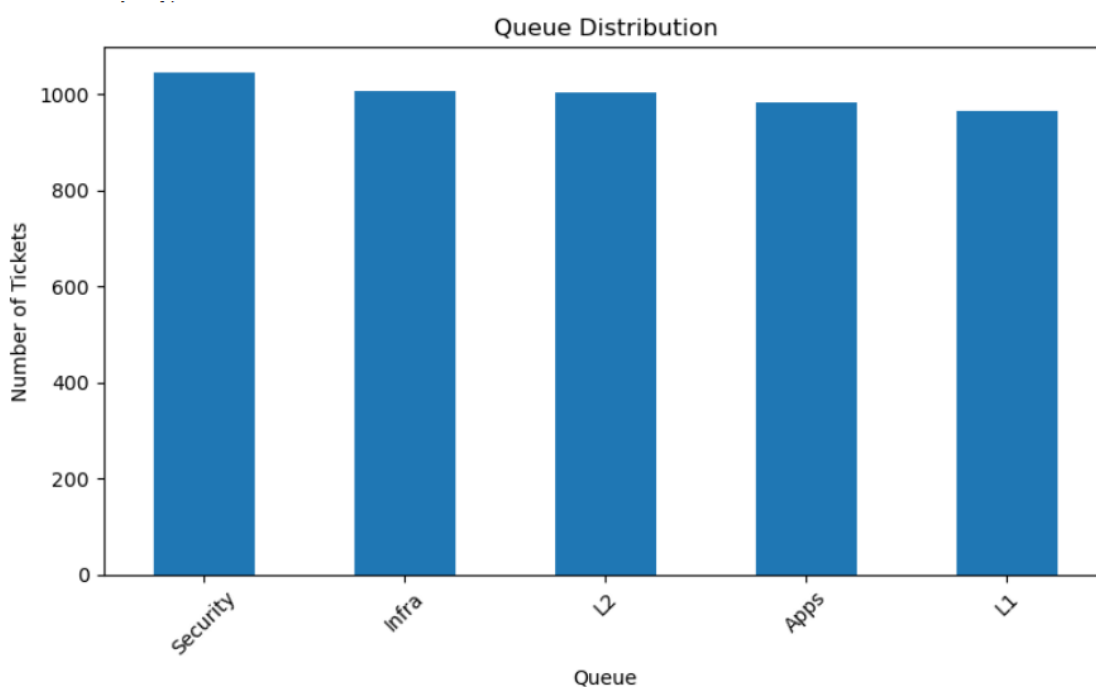


10.8 Queue Distribution

```
] : ## Queue Distribution  
df['Queue'] = df['Team']
```

```
] : queue_counts = df['Queue'].value_counts()  
  
print(queue_counts)  
  
plt.figure(figsize=(8,5))  
queue_counts.plot(kind='bar')  
  
plt.title("Queue Distribution")  
plt.xlabel("Queue")  
plt.ylabel("Number of Tickets")  
  
plt.xticks(rotation=45)  
  
plt.tight_layout()  
plt.show()
```

```
Queue  
Security    1045  
Infra       1005  
L2          1003  
Apps        983  
L1          964  
Name: count, dtype: int64
```



10.9 Priority vs Queue

```
priority_queue = pd.crosstab(df['Priority'], df['Queue'])

print(priority_queue)

priority_queue.plot(kind='bar', figsize=(10,6))

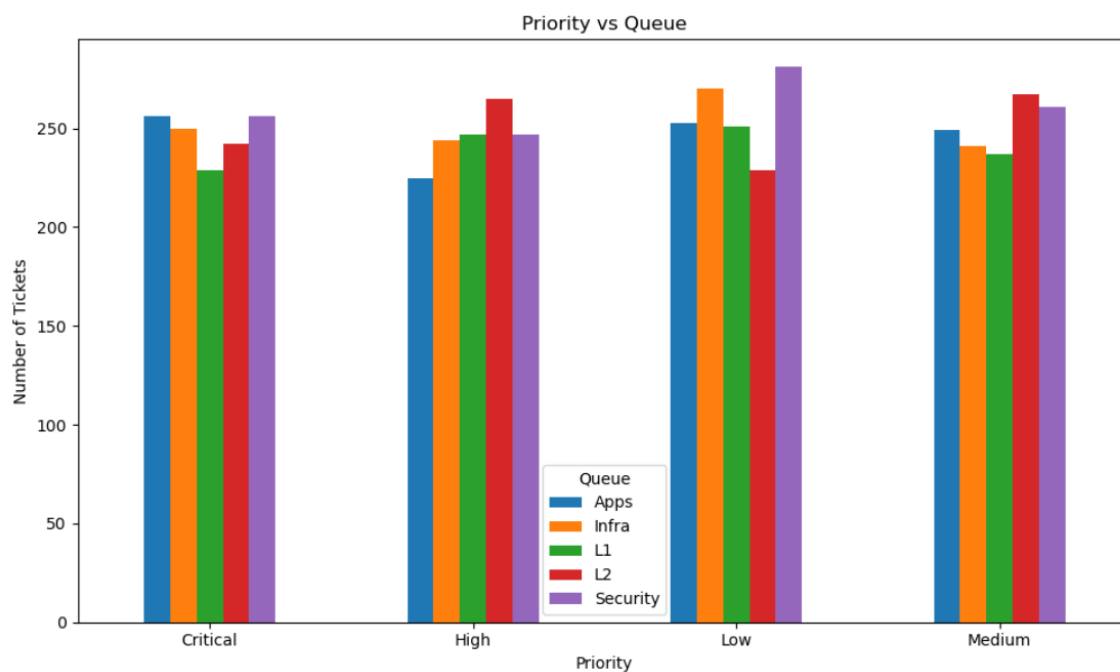
plt.title("Priority vs Queue")
plt.xlabel("Priority")
plt.ylabel("Number of Tickets")

plt.xticks(rotation=0)

plt.legend(title='Queue')

plt.tight_layout()
plt.show()
```

Queue	Apps	Infra	L1	L2	Security
Priority					
Critical	256	250	229	242	256
High	225	244	247	265	247
Low	253	270	251	229	281
Medium	249	241	237	267	261



11. Module 4: Cluster & Similarity Analysis

Include:

- Average Similarity Score by Cluster
- Cluster Size vs Issue Type
- Resolution Time by Cluster
- Resolution Time vs Similarity Score

Key results:

Software cluster highest similarity score = 0.7623.

Email cluster largest = 872 tickets, but its similarity score was lowest among the listed clusters at 0.741.

Important finding:

The analysis identified the Email cluster as an area requiring further review because it combined high ticket volume with comparatively lower similarity

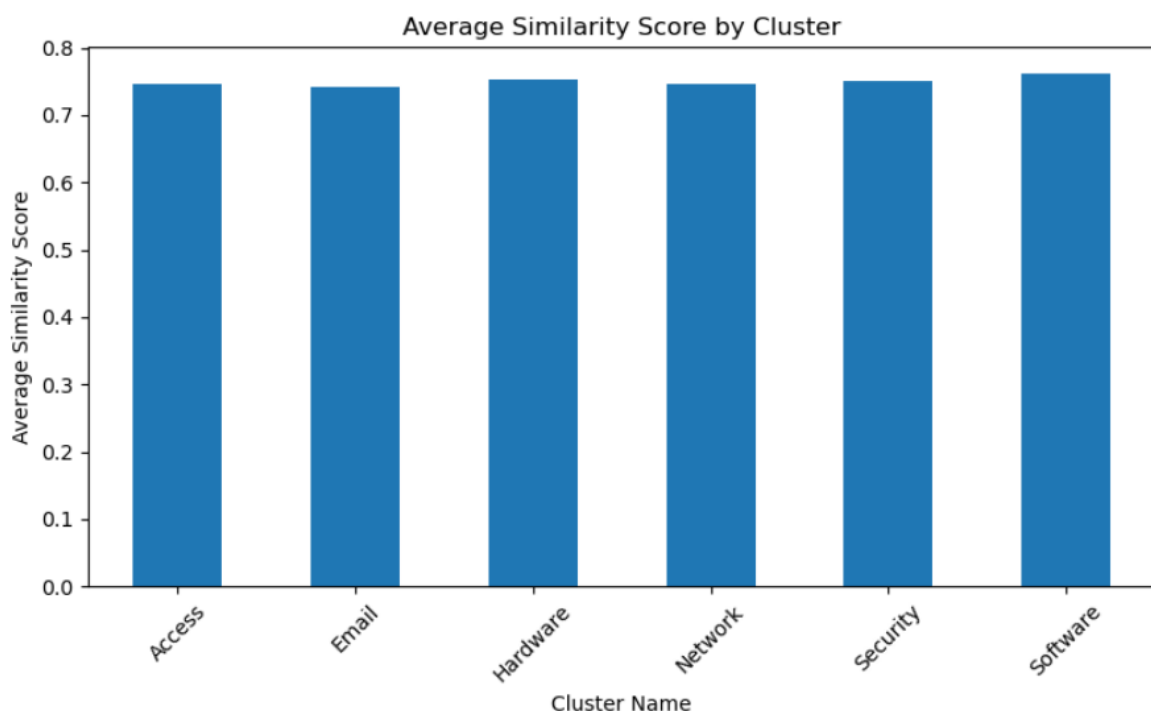
11.1 Average Similarity Score by Cluster

Calculate Average Similarity Score

```
avg_similarity = df.groupby('Cluster_Name')['Similarity_Score'].mean()  
  
print(avg_similarity)
```

```
Cluster_Name  
Access      0.745100  
Email       0.741032  
Hardware    0.752197  
Network     0.746675  
Security    0.750284  
Software    0.762346  
Name: Similarity_Score, dtype: float64
```

```
import matplotlib.pyplot as plt  
  
avg_similarity.plot(kind='bar', figsize=(8,5))  
  
plt.title("Average Similarity Score by Cluster")  
plt.xlabel("Cluster Name")  
plt.ylabel("Average Similarity Score")  
  
plt.xticks(rotation=45)  
plt.tight_layout()  
plt.show()
```



11.2 Cluster Size vs Issue Type

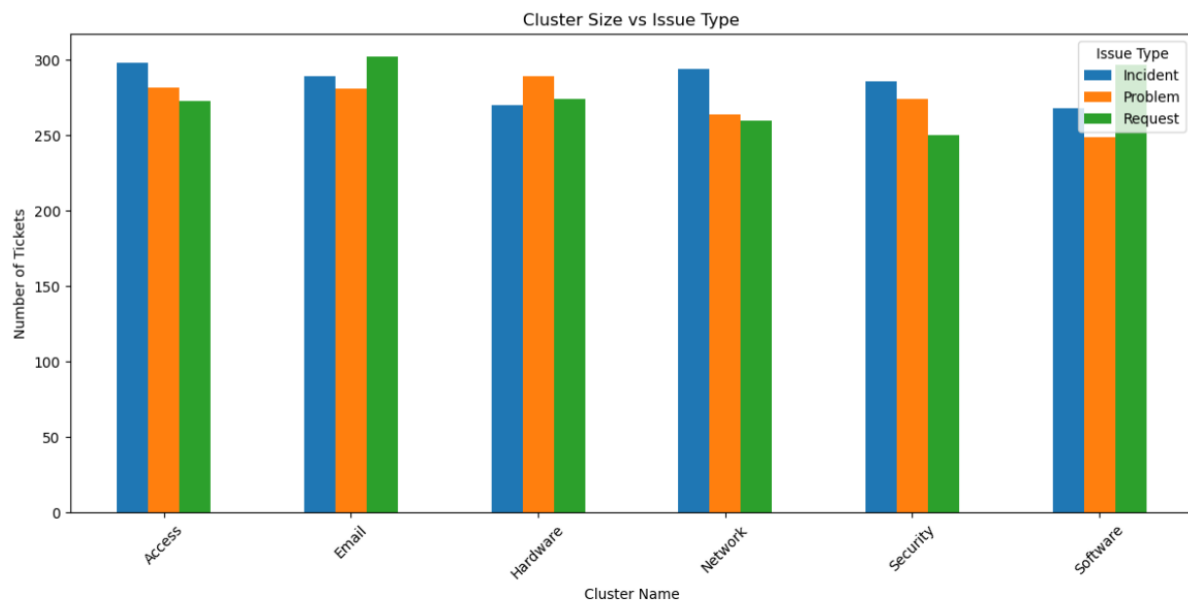
```
import matplotlib.pyplot as plt

cluster_issue.plot(kind='bar', figsize=(12,6))

plt.title("Cluster Size vs Issue Type")
plt.xlabel("Cluster Name")
plt.ylabel("Number of Tickets")

plt.xticks(rotation=45)
plt.legend(title="Issue Type")

plt.tight_layout()
plt.show()
```



11.3 Resolution Time by Cluster

```
import matplotlib.pyplot as plt

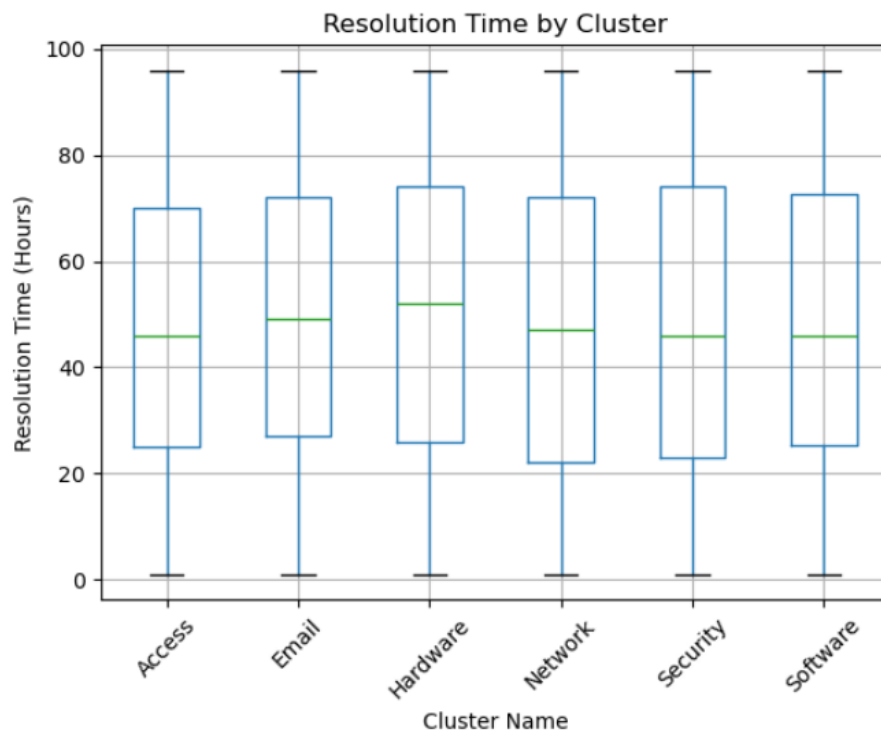
plt.figure(figsize=(10,6))

df.boxplot(column='Resolution_Time', by='Cluster_Name')

plt.title("Resolution Time by Cluster")
plt.suptitle("") # default title remove చేయడానికి
plt.xlabel("Cluster Name")
plt.ylabel("Resolution Time (Hours)")

plt.xticks(rotation=45)
plt.show()
```

<Figure size 1000x600 with 0 Axes>



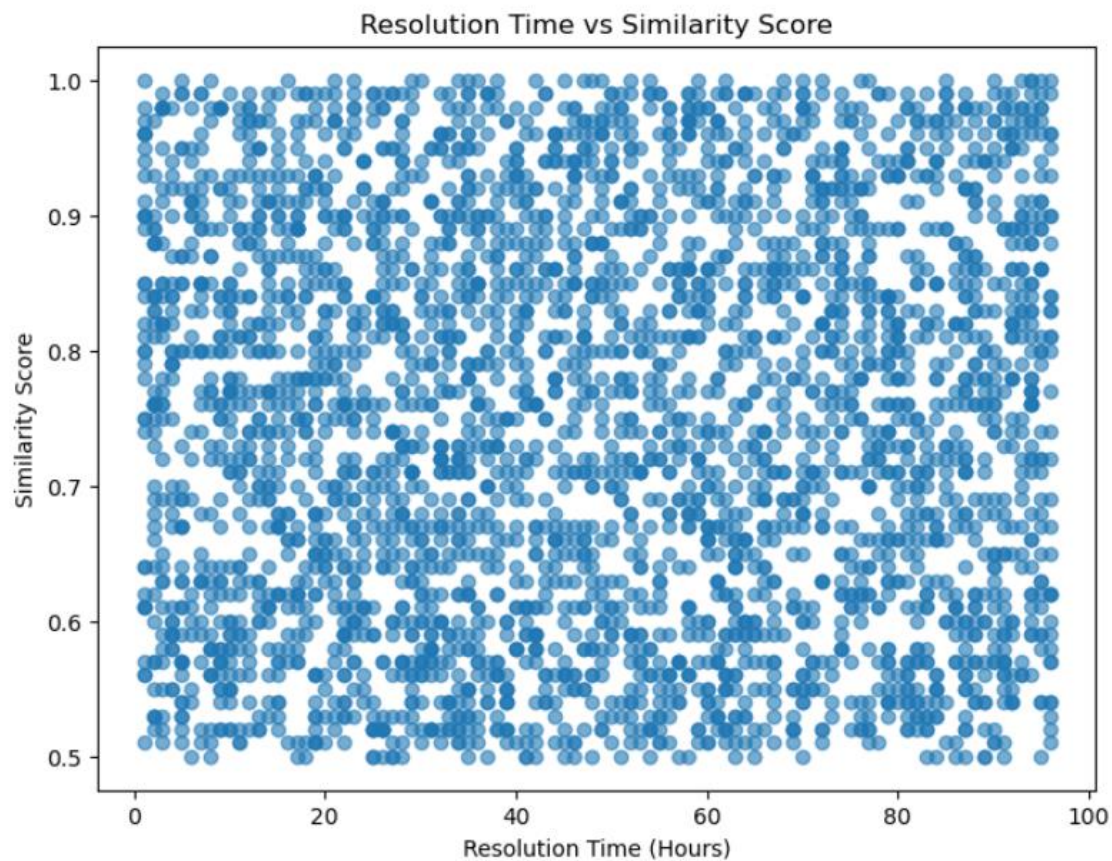
11.4 Resolution Time vs Similarity Score

```
plt.figure(figsize=(8,6))

plt.scatter(
    df['Resolution_Time'],
    df['Similarity_Score'],
    alpha=0.6
)

plt.title("Resolution Time vs Similarity Score")
plt.xlabel("Resolution Time (Hours)")
plt.ylabel("Similarity Score")

plt.show()
```



12. Module 5: Performance Trend Analysis

Include:

1. Average Resolution Time by Priority
2. Average Resolution Time by Ticket Type
3. Fastest Countries
4. Fastest Teams
5. High/Critical Unresolved Tickets
6. Unresolved Tickets by Team

Results:

Analysis	Result
Longest priority resolution	Medium – 49.37 hrs
Fastest ticket type	Problem – 47.51 hrs
Fastest country	Australia – 46.9 hrs
Slowest country	India – 50.02 hrs
High priority open	828
Critical priority open	815
Total High + Critical open	1,643

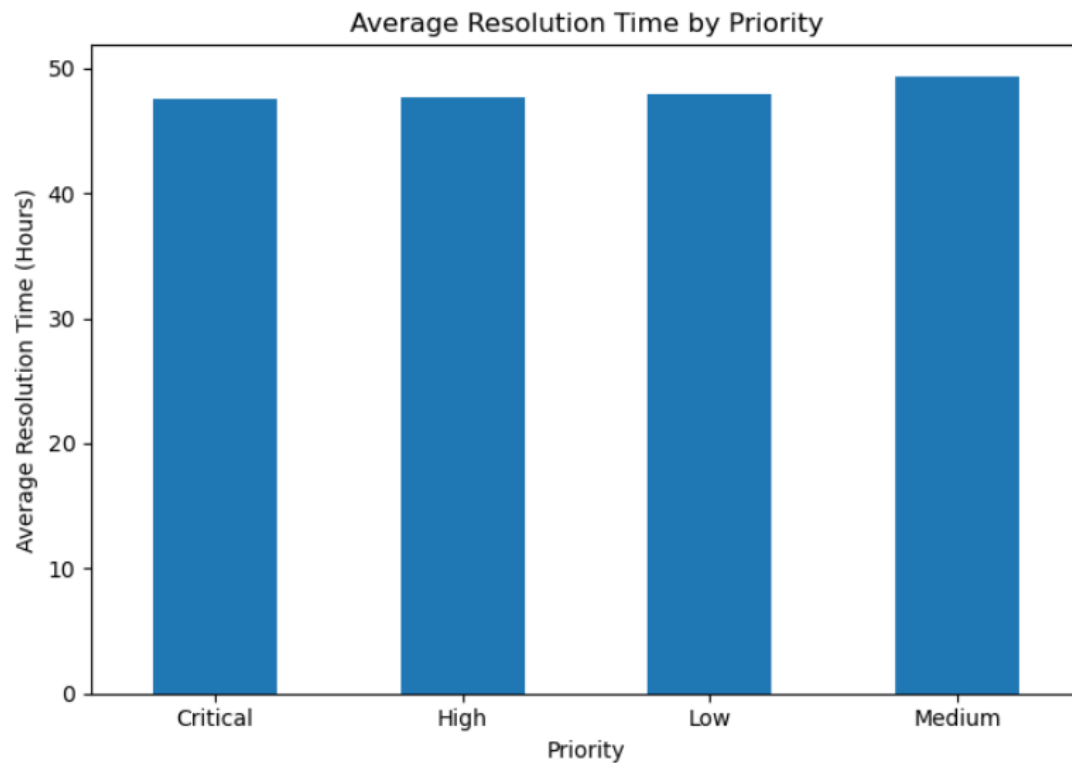
12.1 Average Resolution Time by Priority

```
: import matplotlib.pyplot as plt

avg_priority.plot(kind='bar', figsize=(7,5))

plt.title("Average Resolution Time by Priority")
plt.xlabel("Priority")
plt.ylabel("Average Resolution Time (Hours)")

plt.xticks(rotation=0)
plt.tight_layout()
plt.show()
```

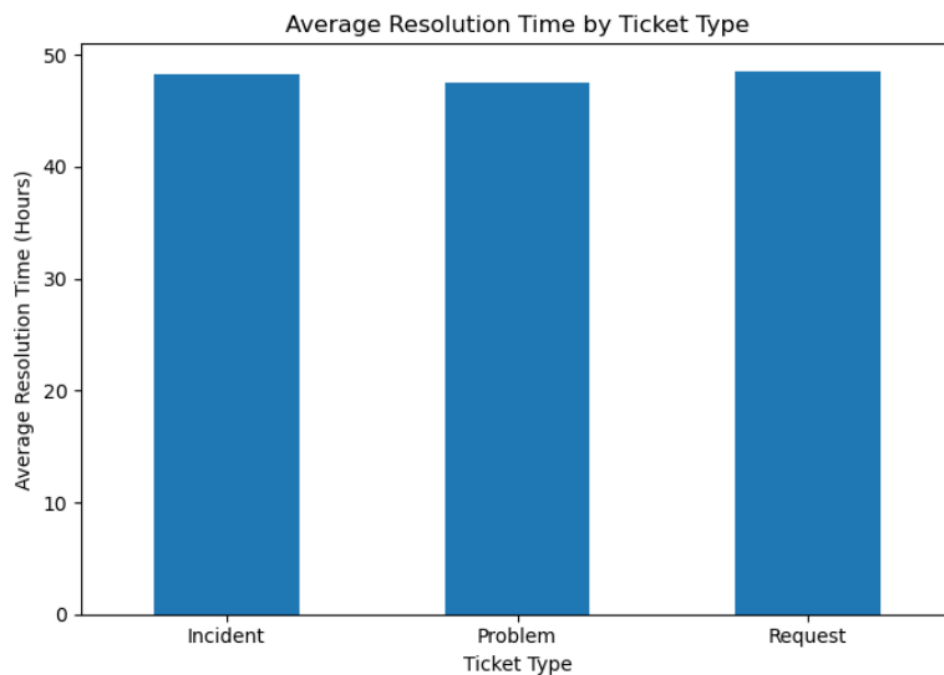


12.2 Average Resolution Time by Ticket Type

```
avg_type.plot(kind='bar', figsize=(7,5))

plt.title("Average Resolution Time by Ticket Type")
plt.xlabel("Ticket Type")
plt.ylabel("Average Resolution Time (Hours)")

plt.xticks(rotation=0)
plt.tight_layout()
plt.show()
```

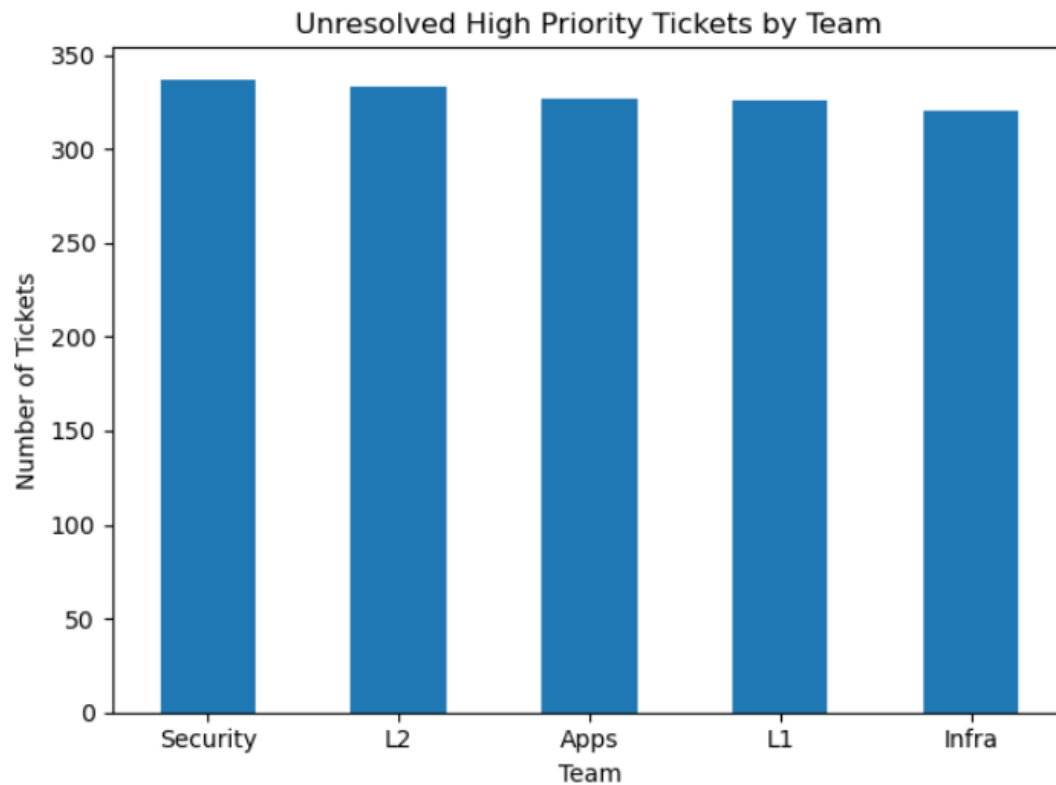


12.3 Unresolved Tickets by Team

```
team_unresolved.sort_values(ascending=False).plot(kind='bar')

plt.title("Unresolved High Priority Tickets by Team")
plt.xlabel("Team")
plt.ylabel("Number of Tickets")

plt.xticks(rotation=0)
plt.tight_layout()
plt.show()
```



13. Module 6: Geographic & Category-Level Insights

13.1 Geographic Data Preparation

The original Supportlytics dataset did not contain **Latitude** and **Longitude** columns. Therefore, I used the available **Region** and **Category** fields to analyze issue categories geographically.

This helps identify which issue categories are more common in each region and supports regional planning.

The coordinates represent the approximate geographic location of each country and are used for country-level visualization only. They do not represent the exact physical location of individual support tickets.

13.2 Regional Ticket Distribution

Region	Tickets
Europe	1,994
APAC	1,989
North America	1,017

Europe and APAC together account for roughly 80% of the ticket volume.

13.3 Geographic Visualization

The generated Latitude and Longitude fields were used to create a geographic visualization in Power BI. This visualization provides a country-level view of support-ticket distribution.



Important:

My original Supportlytics dataset did not contain **Latitude** and **Longitude** columns. Therefore, the available **Region** and **Category** fields were used to analyze issue categories geographically. The geographic analysis represents regional-level patterns rather than exact ticket locations.

13.4 Ticket Concentration Heatmap

A Region × Priority heatmap was created to identify areas of higher ticket concentration and support priority-specific staffing analysis.

```
import matplotlib.pyplot as plt

heatmap_data = df.pivot_table(
    index='Region',
    columns='Priority',
    values='Ticket_ID',
    aggfunc='count',
    fill_value=0
)

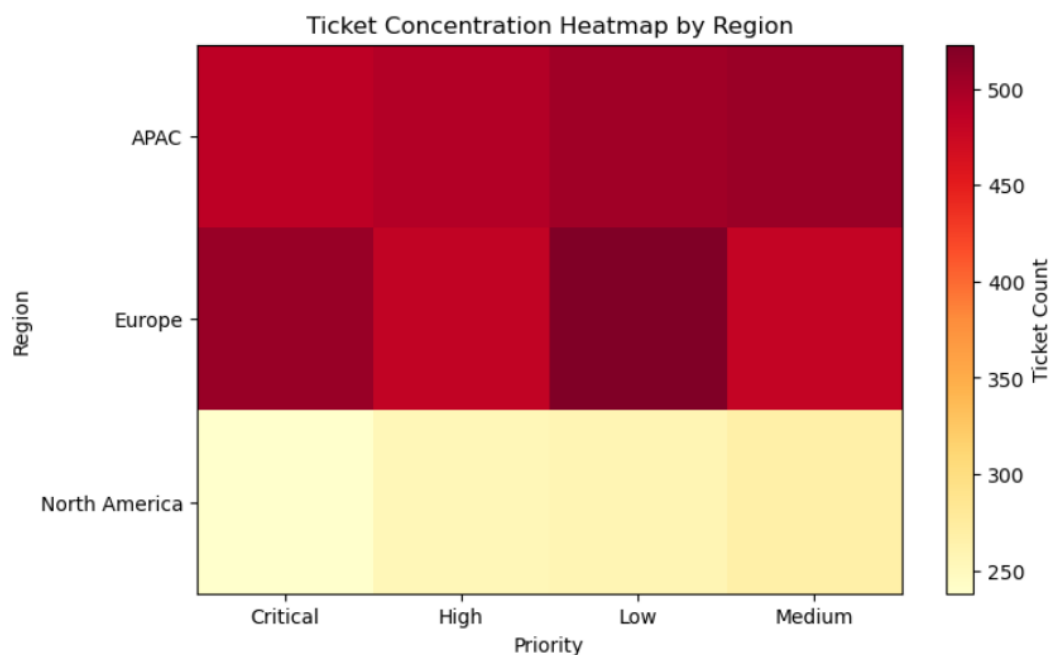
plt.figure(figsize=(8,5))

plt.imshow(heatmap_data, aspect='auto', cmap='YlOrRd')

plt.colorbar(label='Ticket Count')
plt.xticks(range(len(heatmap_data.columns)), heatmap_data.columns)
plt.yticks(range(len(heatmap_data.index)), heatmap_data.index)

plt.title("Ticket Concentration Heatmap by Region")
plt.xlabel("Priority")
plt.ylabel("Region")

plt.show()
```



13.5 Issue Categories by Region

A Region × Category cross-tabulation was used to understand how issue categories vary geographically and to support regional staffing and skill-mix decisions.

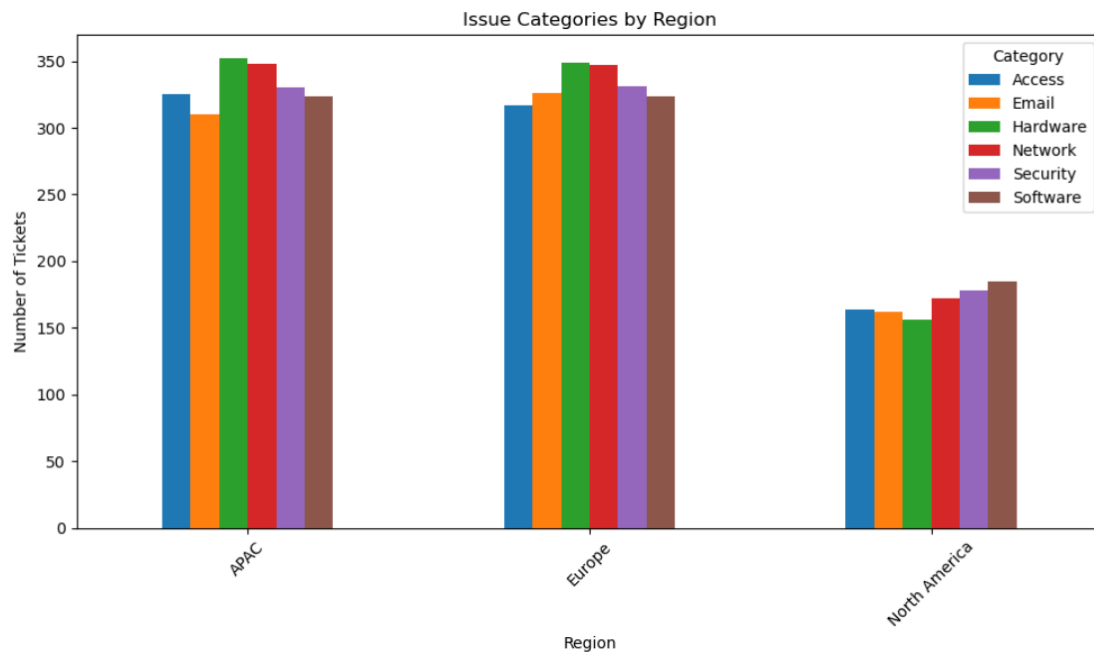
```
category_region = pd.crosstab(df['Region'], df['Category'])

category_region.plot(kind='bar', figsize=(10,6))

plt.title("Issue Categories by Region")
plt.xlabel("Region")
plt.ylabel("Number of Tickets")

plt.xticks(rotation=45)
plt.legend(title="Category")

plt.tight_layout()
plt.show()
```

13.6 Cluster Size vs Performance Score

Finding:

- Network satisfaction ≈ 3.55
- Email cluster size = 872
- Email satisfaction ≈ 3.48
- No clear relationship was observed between cluster size and performance score.

Calculate Average Performance Score

```
performance = df.groupby('Cluster_Name')['Customer_Satisfaction'].mean()
```

```
cluster_performance = cluster_size.to_frame(name='Cluster_Size')
```

```
cluster_performance['Performance_Score'] = performance
```

```
print(cluster_performance)
```

Cluster_Name	Cluster_Size	Performance_Score
Access	853	3.532646
Email	872	3.481802
Hardware	833	3.533454
Network	818	3.549565
Security	810	3.438182
Software	814	3.536900

```
import matplotlib.pyplot as plt

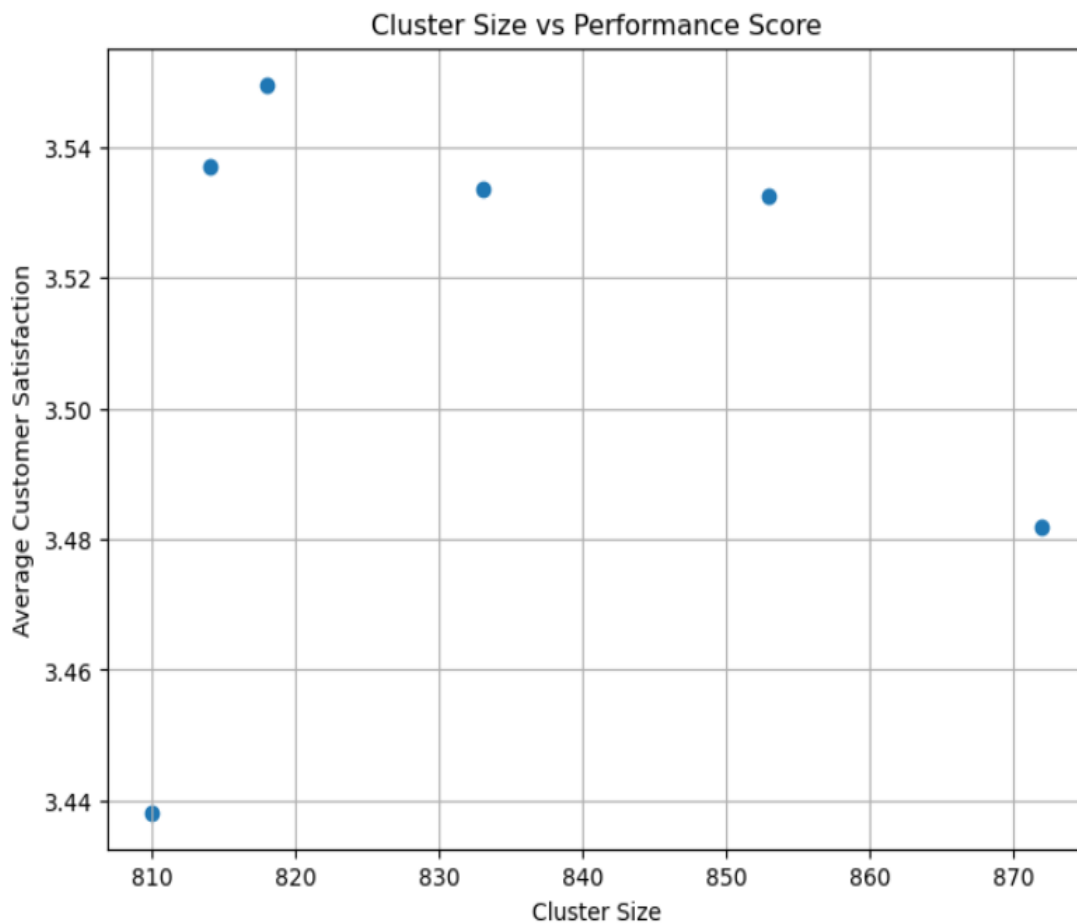
plt.figure(figsize=(8,6))

plt.scatter(
    cluster_performance['Cluster_Size'],
    cluster_performance['Performance_Score']
)

plt.title("Cluster Size vs Performance Score")
plt.xlabel("Cluster Size")
plt.ylabel("Average Customer Satisfaction")

plt.grid(True)

plt.show()
```



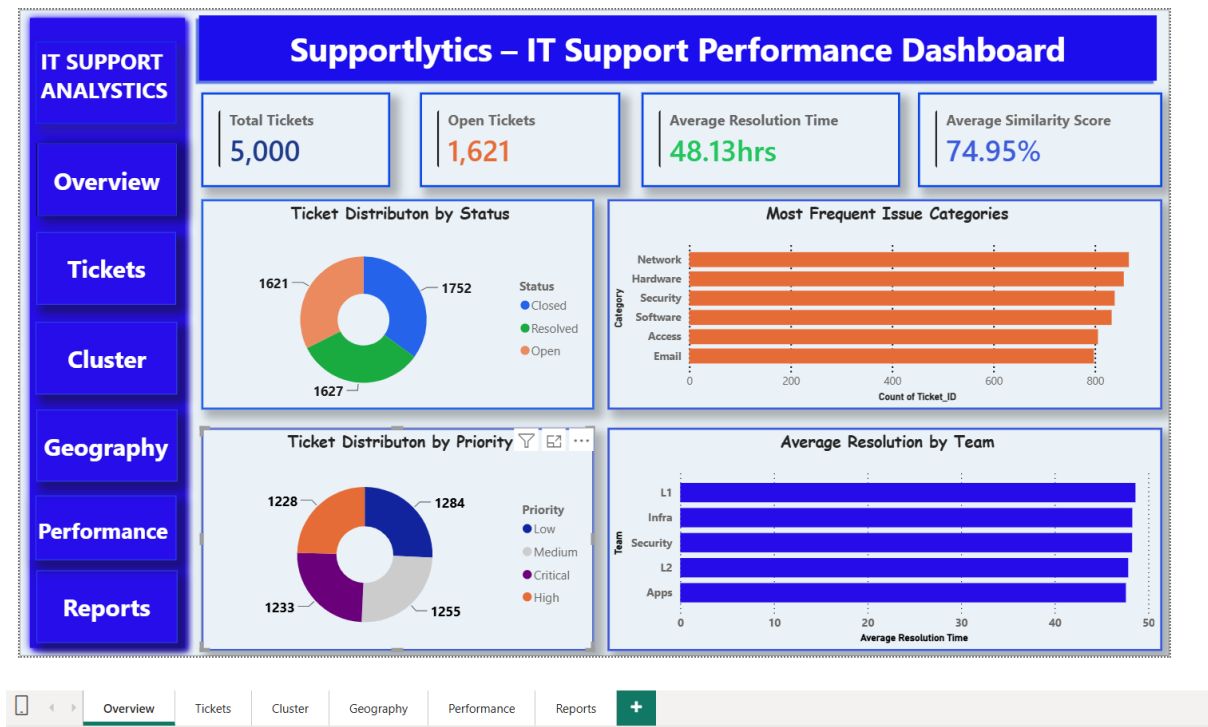
14. Module 7: Power BI Dashboard

14.1 Dashboard Objective

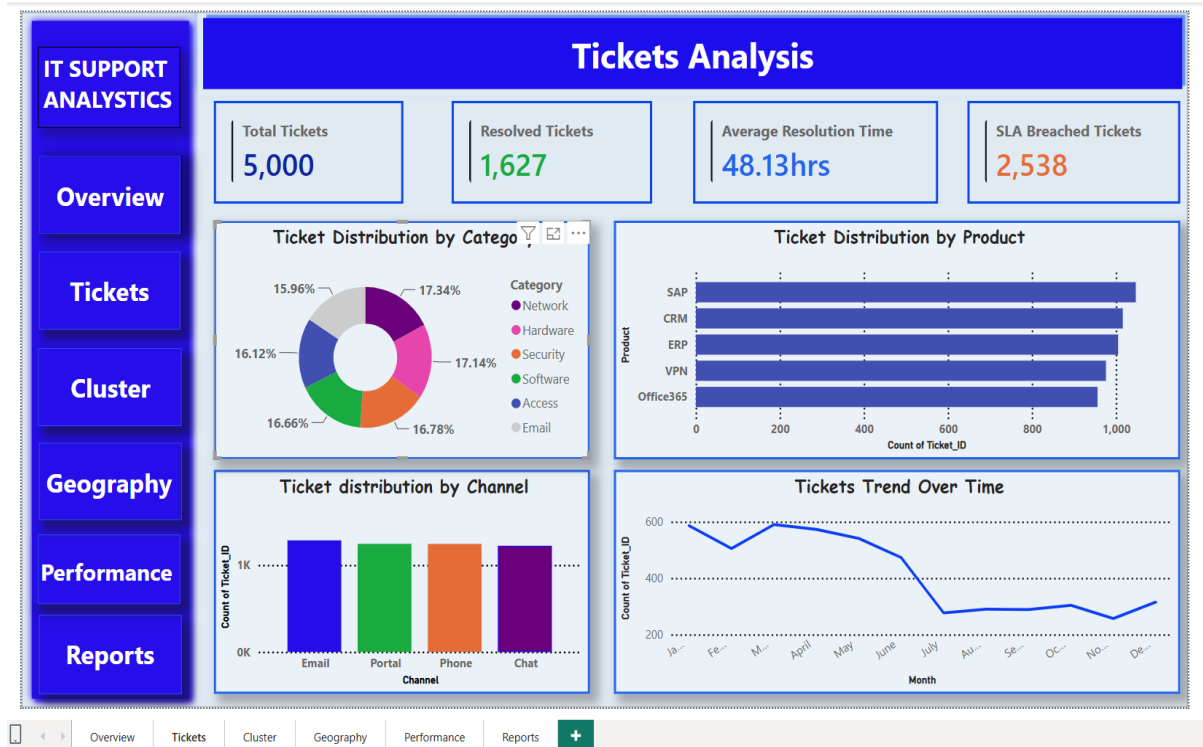
The *Power BI dashboard* was developed to consolidate the analytical findings into an interactive reporting environment.

14.2 Dashboard Components

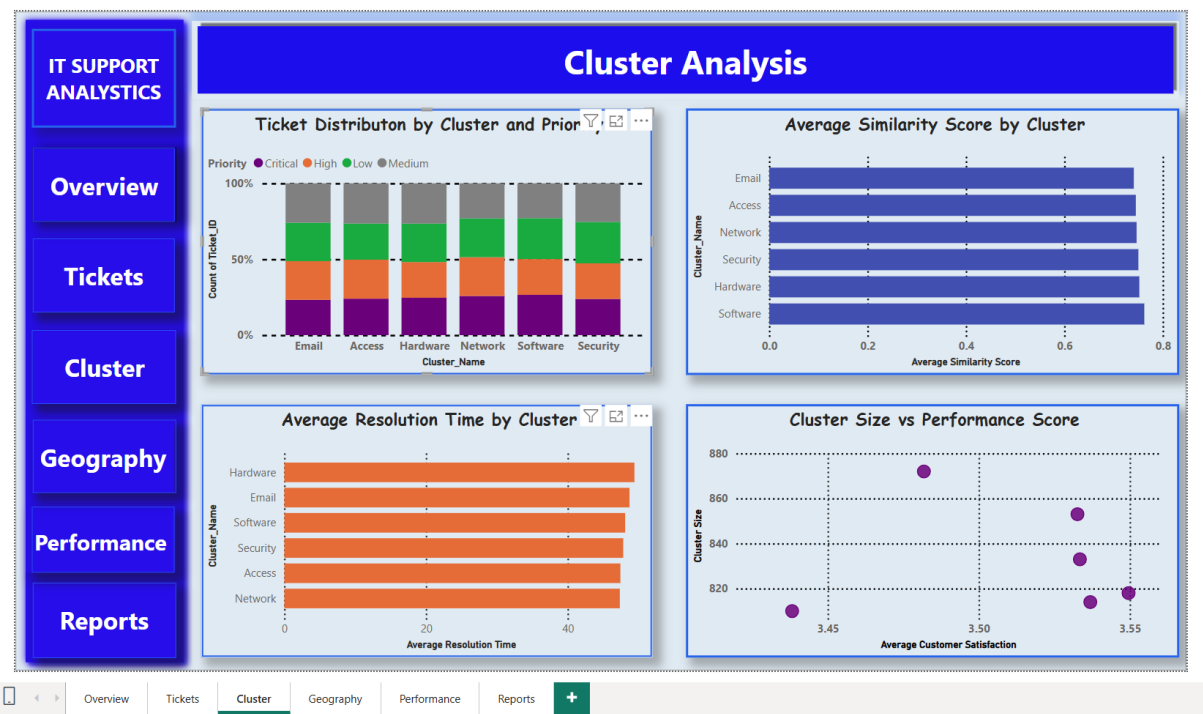
Overview:



Tickets Analysis:



Cluster Analysis:



Performance:



15. Key Findings

1. Ticket types were relatively balanced, with Incident tickets slightly leading.
2. Network and Hardware generated the highest ticket volumes.
3. Software had the highest average cluster similarity score.
4. Email was the largest cluster but had the lowest similarity score among the clusters analyzed.
5. Medium-priority tickets had the longest average resolution time.
6. Australia had the fastest average resolution time among the countries analyzed.
7. 1,643 High/Critical tickets remained unresolved.
8. Security had the largest unresolved high-priority backlog.
9. Europe and APAC generated the majority of ticket volume.
10. Cluster size did not show a clear relationship with performance score.

16. Expected Business Impact

The Supportlytics solution is expected to support IT support teams in making data-driven operational decisions.

Expected benefits:

- Better visibility into ticket workload.
- Faster identification of unresolved high-priority tickets.
- Improved monitoring of resolution performance.
- Better workload allocation across teams and regions.
- Identification of problematic ticket categories/clusters.
- Improved support performance monitoring.
- Data-driven staffing and resource planning.
- Centralized reporting through Power BI.

17. Conclusion

Supportlytics demonstrates an end-to-end approach for analyzing IT support ticket data. The project covered dataset initialization, data quality assessment, cleaning, feature engineering, exploratory visualization, cluster and similarity analysis, performance trend analysis, geographic analysis, and Power BI dashboard development.

The analysis identified important patterns in ticket volume, resolution performance, unresolved high-priority tickets, regional distribution, issue categories, and cluster performance. The resulting dashboard provides a consolidated view of these metrics and can support data-driven decision-making for IT support operations.

Overall, the project demonstrates how data analytics and visualization techniques can transform raw support-ticket data into structured insights and actionable recommendations.